# ALUMNX AI LABS — FDE Intern Hiring Challenge

## The Sales Inbox → Task Router

| | |
|---|---|
| **Format** | Solo. No teams. Open to all. |
| **Submit** | Deployed backend URL (includes your own Task API — see §5) + deployed frontend URL + public GitHub repo, via the submission form |
| **Optional** | A 3-minute video walking through your solution — not required, but a fast way to show your thinking beyond what the code alone conveys |
| **LLM** | Your own Gemini API key (free tier is sufficient) |
| **Updates** | Join the WhatsApp group for kickoff notices and clarifications: https://chat.whatsapp.com/IqvSblln3sbDHO5sHVXFPJ |

---

## 1. The Situation

You've been dropped into a mid-sized B2B services company. Their sales@ inbox receives 150–250 emails a day.

Buried in it are enterprise RFPs worth crores, webinar sponsorship requests with hard deadlines, invoice queries, partnership pitches — and a large volume of noise: SEO spam, newsletters, out-of-office bounces, and "just circling back" replies.

Today, one operations executive reads every email and creates tasks by hand in the company's task management tool. She misses things. RFPs sit for three days. The Marketing lead found out about a conference sponsorship deadline after it had already passed.

Your job: make the inbox route itself — and give the ops team a screen where they can see it working and ask it questions, instead of trusting a black box.

There is no PRD. There is no ticket list. There is a business outcome — emails in, correctly assigned tasks out, nothing important dropped, nothing junk escalated, and a human-readable window into all of it. Engineering the path to it is the assignment.

---

## 2. Your Identity: `candidate_id`

Your Task API is your own — but every task record you create still carries a `candidate_id`, because the grader identifies which submission it's looking at by this field, not by guessing from your URL alone. Your email address, lowercased, is your `candidate_id`.

```
None
candidate_id: "priya.sharma@gmail.com"
```

**Rules — read carefully, this is the #1 cause of unscoreable submissions**

1. Lowercase, trimmed. Your own Task API should normalise on write and read, but don't rely on it upstream — send it clean.
2. Byte-identical everywhere. The email in your submission form, your README, every `POST /tasks` call, and every request your frontend/chat backend makes must be the same string.
3. No `+aliases` in one place and not another. `priya+fde@gmail.com` and `priya@gmail.com` should not be treated as different identities by your own store.
4. Use a real email you check. Results and shortlist notifications go there.

If the grader cannot find tasks under the email you submitted — or your deployed Task API URL doesn't respond — your submission scores zero. There is no manual reconciliation.

---

## 3. What You're Given

### 3.1 `inbox.json` — 250 emails

Real-world messy: HTML fragments, quoted reply chains, forwarded blocks, mixed Hinglish and English, typos, inconsistent signatures, and threads spanning multiple messages.

You can generate 250 similar emails matching this schema, as shown below.

Schema:

JSON


```
{
  "email_id": "em_00142",
  "thread_id": "th_0091",
  "message_index": 0,
  "from_name": "Suresh Kulkarni",
  "from_email": "s.kulkarni@meridiansteel.co.in",
  "to": "sales@company.com",
  "cc": ["procurement@meridiansteel.co.in"],
  "subject": "RFP - Enterprise Document Management System",
  "body": "Dear Team,\n\nPlease find attached our RFP for a
document management system...",
  "received_at": "2026-08-01T09:14:22+05:30",
  "attachments": ["RFP_DMS_2026.pdf"],
  "is_reply": false
}
```


### 3.2 `team_roster.json`

JSON


```
{
  "team": [
    { "user_id": "u_aarti", "name": "Aarti Menon", "department":
"Sales — Enterprise",
      "scope": "RFPs, RFIs, tenders, and inbound deals above
₹10,00,000" },
    { "user_id": "u_rohit", "name": "Rohit Sharma", "department":
"Sales — SMB",
      "scope": "Product enquiries, demo requests, deals at or below
₹10,00,000" },
    { "user_id": "u_meera", "name": "Meera Iyer", "department":
"Marketing",
      "scope": "Webinars, event and conference sponsorships,
content collaborations, PR and media" },
    { "user_id": "u_karan", "name": "Karan Doshi", "department":
"Alliances",
      "scope": "Reseller, channel partner, and technology
integration proposals" },
    { "user_id": "u_divya", "name": "Divya Rao", "department":
"Finance",
```

```
      "scope": "Invoices, purchase orders, payment reminders, GST
and vendor billing" },
    { "user_id": "u_triage", "name": "Triage Queue", "department":
"Operations",
      "scope": "Ambiguous items requiring human review" }
  ]
}
```

### 3.3 Task API

You build and deploy this yourself, to the exact spec in §5. There's no shared, Alumnx-hosted instance — your `/ingest` endpoint writes to your own Task API, and the grader reads your submitted Task API URL directly. No authentication required on the endpoints you expose.

### 3.4 Your own Gemini API key

Free tier is sufficient. Read it from an environment variable. Never hardcode, never commit.

---

## 4. The Routing Rules

Written the way business teams actually write rules — incomplete and slightly contradictory. Handling that gap is part of the test.

| assignee_id | Person | Gets |
|---|---|---|
| u_aarti | Aarti Menon | RFPs, RFIs, tenders, inbound deals above ₹10,00,000 |
| u_rohit | Rohit Sharma | Product enquiries, demo requests, deals at or below ₹10,00,000 |
| u_meera | Meera Iyer | Webinars, event/conference sponsorships, content collaborations, PR and media |
| u_karan | Karan Doshi | Reseller, channel partner, technology integration proposals |
| u_divya | Divya Rao | Invoices, POs, payment reminders, GST and vendor billing |

| | | |
|---|---|---|
| `u_triage` | Triage Queue | Anything ambiguous or that doesn't cleanly fit above |

Additional rules:

1. Any email with a stated deadline within 72 hours of `received_at` is `priority: "high"`, regardless of owner.
2. A reply on an existing thread must **update** the existing task — not create a second one.
3. Government and PSU tenders always go to Aarti, irrespective of deal value.
4. Do not create tasks for out-of-office auto-replies, newsletters, or unsolicited vendor spam.

These rules will not cover every email in the dataset. Some are genuinely ambiguous; some pit two rules against each other. **How you handle the emails the rules don't cover matters more to us than how you handle the ones they do.**

---

# 5. The Task API — Build and Deploy This Yourself

This is not a service we host — it's a spec you implement as part of your own backend (§7.2) and deploy publicly. The routes, fields, and enums below are exact; the grader calls your deployed URL with these same shapes. All requests and responses are `application/json`. No auth headers.

**Two things that will silently cost you points if skipped:**

- **One base URL, all routes.** Everything in this section (`/tasks`, `/users`) and everything in §7 (`/ingest`, `/api/tasks`, `/api/stats`, `/api/chat`) must be reachable under the single backend URL you submit. Don't split the Task API onto a separate deployment with its own URL — the grader only calls the one URL from your submission form.
- **Real persistence, not in-memory state.** Runs 2 and 3 (§8.1) happen as separate requests, possibly minutes apart, and Run 3 specifically depends on Run 1's tasks still existing. If you store tasks in a plain in-process array or dict, a cold restart on free-tier hosting (Render, Railway free tiers spin down after ~15 minutes idle) will wipe your data between runs and you'll fail idempotency and thread reconciliation through no fault of your routing logic. Use SQLite, Postgres, or any store that survives a restart.

If you don't already have a stack in mind: **FastAPI** for the API, hosted free on **Render**, with **Supabase** (free Postgres) for persistence, is a solid default that satisfies everything above. Not required — any combination that meets the spec works equally well.

## 5.1 `POST /tasks` — create a task

JSON

```
{
  "candidate_id": "priya.sharma@gmail.com",
  "source_email_id": "em_00142",
  "thread_id": "th_0091",
  "title": "RFP — Enterprise DMS for Meridian Steel",
  "description": "Meridian Steel has issued an RFP for a document
management system. Submission due 12 Aug 2026.",
  "assignee_id": "u_aarti",
  "category": "enterprise_rfp",
  "priority": "high",
  "due_date": "2026-08-12",
  "deal_value_inr": 2500000,
  "company_name": "Meridian Steel Pvt Ltd",
  "confidence": 0.91
}
```

Response `201`:

JSON

```
{
  "task_id": "tsk_8f2a1c",
  "candidate_id": "priya.sharma@gmail.com",
  "source_email_id": "em_00142",
  "created_at": "2026-08-09T11:02:14+05:30"
}
```

Error `400`:

JSON

```
{
  "error": "invalid_enum_value",
  "field": "assignee_id",
  "received": "Aarti",
  "allowed": ["u_aarti", "u_rohit", "u_meera", "u_karan",
"u_divya", "u_triage"]
}
```

Nobody but you enforces this on the way in — it's your own API, so nothing external stops you from accepting a malformed payload. Implement this validation anyway: we will hit your `/tasks`

endpoint directly with a bad enum during grading and expect this exact error shape back. Skipping it isn't a shortcut, it's a point loss.

### 5.2 Field reference

| Field | Type | Required | Notes |
|---|---|---|---|
| `candidate_id` | string | Yes | Your lowercased email. Wrong value = unscoreable. |
| `source_email_id` | string | Yes | The `email_id` that produced this task. |
| `thread_id` | string | Yes | From the source email. |
| `title` | string | Yes | Free text. Not machine-scored. |
| `description` | string | No | Free text. Not machine-scored. |
| `assignee_id` | enum | Yes | `u_aarti` · `u_rohit` · `u_meera` · `u_karan` · `u_divya` · `u_triage` |
| `category` | enum | Yes | `enterprise_rfp` · `smb_enquiry` · `marketing` · `alliances` · `finance` · `triage` |
| `priority` | enum | Yes | `high` · `medium` · `low` |
| `due_date` | string \| null | Yes | `YYYY-MM-DD`. `null` if not stated. |
| `deal_value_inr` | integer \| null | Yes | Rupees, no decimals. `null` if not stated or inferable. |
| `company_name` | string \| null | Yes | `null` if not determinable. |
| `confidence` | float | Yes | 0.0–1.0. Your own certainty in this routing decision. |

Enums are exact-match and case-sensitive. Fabricated `due_date`, `deal_value_inr`, or `company_name` scores worse than leaving the field `null`.

### 5.3 `PATCH /tasks/{task_id}` — update an existing task

Accepts any subset of: `title`, `description`, `assignee_id`, `category`, `priority`, `due_date`, `deal_value_inr`, `company_name`, `confidence`. Returns `200` with the full updated task.

### 5.4 `GET /tasks?candidate_id={email}` — list your tasks

`candidate_id` is mandatory. Optional filters: `&thread_id=`, `&source_email_id=`, `&assignee_id=`.

Note this is a different route from `GET /api/tasks` in §7.2. `/tasks` is the raw Task API spec — this is what the grader calls directly to score you. `/api/tasks` is your own backend's wrapper for your frontend, which can add fields the grader doesn't care about (like why something was skipped). Build both; don't merge them into one route, or the grader's lookup in §8.1 won't find what it expects.

### 5.5 `DELETE /tasks/{task_id}` — delete one task

For cleanup during development. No bulk delete.

### 5.6 `GET /users` — the team roster

Should return the same `team` array as `team_roster.json` (§3.2) — this isn't separately scored, but your chat interface (§7.3) will need to resolve `assignee_id` to a name somewhere, and this is the obvious place to serve it from rather than hardcoding names in your frontend.

> ⚠️ **Nothing in this spec forces you to protect yourself from yourself.** `POST /tasks` has no required uniqueness constraint — if you don't build deduplication into your own Task API, posting the same `source_email_id` five times gives you five tasks, with no 409 stopping you. Preventing duplicates is entirely your responsibility, and we test it directly (§8).

---

## 6. Worked Examples

These twelve cases show exactly what's expected. Study them — most of the difficulty in the real dataset is a variation on one of these.

### Example 1 — Clean enterprise RFP

Body: *"Meridian Steel invites proposals for an enterprise DMS covering 4 plants and ~1,200 users. Indicative budget is Rs. 25 lakhs. Proposals must reach us by 12th August 2026."* Received 2026-08-01.

Expected task:

```json
{
  "assignee_id": "u_aarti",
  "category": "enterprise_rfp",
  "priority": "medium",
  "due_date": "2026-08-12",
  "deal_value_inr": 2500000,
  "company_name": "Meridian Steel"
}
```

Why: ₹25 lakhs > ₹10 lakh threshold → Aarti. Deadline is 11 days out, not within 72 hours → medium, not high. Note "Rs. 25 lakhs" must be parsed to 2500000.

### Example 2 — SMB demo request, no value stated

Body: *"Hi, we're a 30-person logistics startup in Pune... can we get a demo sometime next week? Nothing urgent. — Ankit Bose, Founder, Railyard Logistics"*

Expected task:

```json
{
  "assignee_id": "u_rohit",
  "category": "smb_enquiry",
  "priority": "low",
  "due_date": null,
  "deal_value_inr": null,
  "company_name": "Railyard Logistics"
}
```

Why: Small company, demo request, no stated value → Rohit. `deal_value_inr` is null, not a guess. "Sometime next week" is not a deadline → `due_date: null`. "Nothing urgent" → low.

### Example 3 — PSU tender below the threshold

Body: *"Tender Notice No. BHEL/PROC/2026/0847. Bharat Heavy Electricals Limited invites bids for supply of analytics software licences. Estimated value: Rs. 6,50,000. Last date for bid submission: 03-08-2026, 1700 hrs IST."* Received 2026-08-01, 14:20 IST.

Expected task:

```json
{
  "assignee_id": "u_aarti",
  "category": "enterprise_rfp",
  "priority": "high",
  "due_date": "2026-08-03",
  "deal_value_inr": 650000,
  "company_name": "Bharat Heavy Electricals Limited"
}
```

Why: ₹6.5 lakhs is below the threshold, which points to Rohit — but Rule 3 overrides Rule 1: PSU tenders always go to Aarti. Deadline is ~51 hours away → high. This example exists specifically to catch systems that apply the value rule blindly.

### Example 4 — Marketing sponsorship, hard deadline

Body: *"We're finalising sponsors for the India SaaS Summit in Bengaluru. Gold tier is ₹4,00,000 and includes a keynote slot. We need confirmation by tomorrow EOD as we're going to print. — Nandita Reddy, Sponsorship Lead"* Received 2026-08-02, 16:45 IST.

Expected task:

```json
{
  "assignee_id": "u_meera",
  "category": "marketing",
  "priority": "high",
  "due_date": "2026-08-03",
  "deal_value_inr": 400000,
  "company_name": "India SaaS Summit"
}
```

Why: Event sponsorship → Meera, not Sales, even though money is involved and a number is stated. "Tomorrow EOD" is within 72 hours → high. This is exactly the failure the ops team complained about.

### Example 5 — Finance

Body: *"Please find attached invoice INV-2026-0331 for Rs. 1,18,000 (incl. 18% GST) against PO-88214. Kindly process — payment terms were Net 30 and this is now 12 days overdue. Also, our GSTIN has changed, updated details attached."*

Expected task:

```json
{
  "assignee_id": "u_divya",
  "category": "finance",
  "priority": "high",
  "due_date": null,
  "deal_value_inr": null,
  "company_name": "Vantage Cloud Services"
}
```

Why: Invoice → Divya. `deal_value_inr` is null — ₹1,18,000 is an invoice amount, not a deal value. Overdue payment justifies high despite no explicit date. No specific date stated → `due_date: null`.

### Example 6 — Alliances

Body: *"We're a Salesforce implementation partner across MEA with 40+ enterprise clients. We'd like to explore reselling your platform in the region, or a technical integration at minimum. Who handles partnerships?"*

Expected task:

```json
{
  "assignee_id": "u_karan",
  "category": "alliances",
  "priority": "medium",
  "due_date": null,
  "deal_value_inr": null,
  "company_name": "Zenith Cloud Partners"
}
```

Why: Reseller/channel language → Karan. Tempting to route to Sales because it mentions clients and revenue potential — it isn't a deal.

### Example 7 — Out-of-office (NO TASK)

Body: *"I am out of office until 14th August with limited access to email. For urgent matters please contact my colleague at raghav@northbridge.in. — Sent from Outlook"*

Expected: No task created.

Why: Rule 4. An auto-reply is not work. Systems that create a `u_triage` task here are marked spurious — the whole point is to reduce the ops executive's queue, not relocate it.

### Example 8 — Vendor spam disguised as marketing (NO TASK)

Body: *"Hi, I noticed your website isn't ranking on page 1 for key terms. We've helped 200+ SaaS companies 3x their organic traffic. We do content marketing, PR outreach, and webinar promotion. Free audit attached — interested in a quick 15 min call?"*

Expected: No task created.

Why: Contains every Marketing keyword — content, PR, webinar — and is unsolicited vendor spam. This is the single most common failure. Keyword matching sends this to Meera. Understanding direction of intent (they're selling to us, not buying from us) is what separates a working system from a demo.

### Example 9 — Newsletter (NO TASK)

Body: *"The B2B Growth Weekly — Issue #212. In this edition: why PLG is stalling, 5 pricing experiments that worked, and a teardown of Figma's onboarding. [Unsubscribe]"*

Expected: No task created. Rule 4.

### Example 10 — Thread reply (PATCH, NOT POST)

Follow-up on Example 1, same `thread_id: "th_0091"`. Body: *"Correction to our earlier note — the board has approved an increased budget of Rs. 32 lakhs, and the submission deadline is advanced to 11th August. Apologies for the change."* (plus quoted original text below it). Received 2026-08-09.

Expected: `PATCH /tasks/tsk_8f2a1c`

```json
{
  "priority": "high",
  "due_date": "2026-08-11",
```

```
  "deal_value_inr": 3200000
}
```

Why: Same thread → update, never create. Deadline is now ~48 hours out → escalate to high. No new task. Also note the quoted original text in the body — don't re-extract from it and double-count.

### Example 11 — Genuinely ambiguous (TRIAGE)

Body: *"Hi — we met at your booth in Mumbai. Two things: (1) we'd like to evaluate your platform for our 800-person org, budget TBD but likely significant, and (2) our CMO wants to co-host a webinar with your team in September. Can you loop in the right people? — Farhan Qureshi, VP Strategy, Halcyon Retail"*

Expected task:

JSON

```
{
  "assignee_id": "u_triage",
  "category": "triage",
  "priority": "medium",
  "due_date": null,
  "deal_value_inr": null,
  "company_name": "Halcyon Retail",
  "confidence": 0.42
}
```

Why: Two distinct asks owned by two different people, and "budget TBD" means the ₹10 lakh rule cannot be applied. Routing to `u_triage` with a low confidence and a clear reason in `description` is the correct answer — better than confidently picking one and dropping the other. Splitting into two tasks is a defensible alternative; say so in `DECISIONS.md` if you do.

### Example 12 — Hinglish, informal, value in shorthand

Body: *"Bhai, humko aapka product chahiye for our dealer network. Around 150 users honge. Budget approx 1.2 cr allocated hai for this FY. Kab connect kar sakte hain? Thoda jaldi, board review 20th ko hai."* Received 2026-08-05.

Expected task:

```json
{
  "assignee_id": "u_aarti",
  "category": "enterprise_rfp",
  "priority": "medium",
  "due_date": "2026-08-20",
  "deal_value_inr": 12000000,
  "company_name": null
}
```

Why: "1.2 cr" → 12000000. Well above threshold → Aarti. 20th is 15 days out → medium, not high. `company_name` is null — no company is named anywhere. Do not infer one from the email domain unless it is unambiguous.

### 6.1 Quick reference

| # | Signal | Route | Trap |
|---|---|---|---|
| 1 | RFP, ₹25L | `u_aarti` | Parse "lakhs" |
| 2 | Demo, no value | `u_rohit` | Don't guess a value |
| 3 | PSU tender, ₹6.5L | `u_aarti` | Rule 3 beats Rule 1 |
| 4 | Sponsorship, ₹4L | `u_meera` | Money ≠ Sales |
| 5 | Invoice | `u_divya` | Invoice amount ≠ deal value |
| 6 | Reseller | `u_karan` | Not a deal |
| 7 | Auto-reply | none | Triage is not a dumping ground |
| 8 | SEO spam | none | Direction of intent |
| 9 | Newsletter | none | — |
| 10 | Thread reply | PATCH | Ignore quoted text |
| 11 | Two asks | `u_triage` | Low confidence is correct |

| | | | |
|---|---|---|---|
| 12 | Hinglish, "1.2 cr" | `u_aarti` | Don't invent a company |

---

# 7. What You Must Ship

`/ingest` alone is necessary but not sufficient — the ops executive needs somewhere to *look*, and she needs to be able to *ask*.

## 7.1 `POST /ingest` — the required endpoint

```
JSON
{
  "candidate_id": "priya.sharma@gmail.com",
  "emails": [
    { "email_id": "em_00301", "thread_id": "th_0150", "...": "..."
}
  ]
}
```

Response `200`:

```
JSON
{ "processed": 60, "tasks_created": 41, "tasks_updated": 7,
"skipped": 12, "errors": [] }
```

Must be synchronous — return `200` only after every task has actually been written to the Task API. Batches of up to 100. Timeout: 15 minutes per batch.

## 7.2 A backend service (required)

You must stand up your own backend, which now includes your own Task API (§5) as one of its concerns. It is the thing your conversational interface talks to — **neither should call Gemini directly from the browser, and the browser should never talk to your Task API directly either.** At minimum it exposes:

| Endpoint | Purpose |
|---|---|
| `POST /ingest` | As in §7.1. |

| | |
|---|---|
| `GET /api/tasks` | Reads from your own Task API (§5), optionally joined with your own stored classification metadata (e.g. why something was skipped, since the Task API spec itself has no concept of "skipped"). |
| `GET /api/stats` | Aggregate counts: processed / created / updated / skipped / spurious-flagged, broken down by category and by run. This is what your `/api/stats` responses and chat answers read from. |
| `POST /api/chat` | The conversational endpoint — see §7.3. |

Any backend stack is fine (Node/Express, FastAPI, Flask, Hono, whatever you're fastest in). The requirement is architectural, not technological: **your Gemini key and any local database must never be reachable from client-side JS.**

Your backend needs somewhere to persist what the Task API doesn't track — specifically, *skipped* emails (out-of-office, newsletter, spam) never become tasks at all, but the ops executive still needs to see that they were seen and correctly ignored. Store enough about every email you process (decision, category, confidence, reasoning) to answer the chat interface without re-calling Gemini for facts you already know.

### 7.3 A conversational interface (required)

This is the frontend deliverable — not a full reviewer dashboard, just one focused surface: **paste emails in, see them, ask questions about them.** It has three parts in this order. Any frontend framework works; **React** is a solid default if you don't already have a preference.

**1. JSON input.** A text area (or file upload) where a batch of emails — same schema as `inbox.json`, any size up to the 100-per-batch ingest limit — can be pasted or dropped in directly.

**2. Renders as a table.** On submit, before anything is routed, show the pasted batch as a **plain data table** — one row per email, columns for `from_name`, `from_email`, `subject`, `received_at`, `thread_id`, and a truncated `body` preview. This is a raw view of what came in, independent of your routing logic, so a grader can visually sanity-check the input before trusting any output derived from it.

Under the input box, add an option to generate 250 similar sample emails matching this schema, for anyone trying the interface without pasting their own batch.

**Sample layout — raw JSON input rendered as a table** (for reference — your visual design can differ, the information architecture should not):

| from_name | from_email | subject | received_at | thread_id |
|---|---|---|---|---|
| Suresh Kulkarni | s.kulkarni@meridiansteel.co.in | RFP - Enterprise Document Management System | 2026-08-01 09:14 | th_0091 |
| Ankit Bose | ankit@railyardlogistics.in | Quick demo request | 2026-08-01 11:02 | th_0092 |
| Nandita Reddy | nandita@saassummit.in | Sponsorship confirmation needed | 2026-08-02 16:45 | th_0093 |
| — (auto-reply) | s.kulkarni@meridiansteel.co.in | Out of Office | 2026-08-03 08:00 | th_0091 |

**3. Ask questions about it.** A chat panel where the ops executive can ask natural-language questions about the batch she just pasted (or generated). It must be able to answer things like:

- "How many emails this batch were proposal or RFP-related?"
- "How many were marketing versus actual spam we correctly ignored?"
- "Show me everything sitting in triage and why."
- "What's our spurious rate so far?"
- "Which high-priority tasks are still unassigned-feeling — i.e., low confidence?"

**This must be grounded, not vibes.** The chat backend should translate the question into a structured query over your own stored processing data (§7.2) — counts, filters, group-bys — and have Gemini phrase the *answer*, not invent the *numbers*. A chat interface that hallucinates a plausible-sounding count is worse than no chat interface at all, and we will ask questions in the live defence specifically designed to catch this (e.g. asking about a category that had zero emails, and checking whether it confidently invents a number instead of saying zero).

`POST /api/chat` request/response shape (yours to design, but must look roughly like this):

```json
// request
{ "candidate_id": "priya.sharma@gmail.com", "query": "How many marketing vs RFP emails came in?" }

// response
{
```

```
  "answer": "14 emails were routed as enterprise_rfp and 9 as
marketing. 3 additional emails used marketing keywords but were
correctly skipped as vendor spam.",
  "supporting_data": {
    "enterprise_rfp": 14,
    "marketing": 9,
    "skipped_marketing_lookalike_spam": 3
  }
}
```

Returning `supporting_data` alongside `answer` is required — it's how we check the answer is actually backed by a query result and not free-floating text.

Sample chat exchange, scoped to the pasted batch above:

> **Ops exec:** Of the emails I just pasted, how many look like proposals versus marketing? **System:** Of these 4 emails: 1 looks like an enterprise RFP (Meridian Steel), 1 is a marketing sponsorship ask (SaaS Summit), 1 is an SMB demo request, and 1 is an out-of-office auto-reply that wouldn't generate a task.

**Sample questions and expected answers.** These are representative of what we'll actually type into your deployed chat interface during grading — including the two traps (a zero-count category and an out-of-scope ask) that most naive implementations fail.

| # | Sample question | Expected answer behavior | `supporting_data` shape |
|---|---|---|---|
| 1 | "How many emails this batch were proposal or RFP related?" | States the count for `enterprise_rfp`, pulled from stored classification data, not re-inferred from raw email text. | `{ "enterprise_rfp": 14 }` |
| 2 | "How many were marketing versus actual spam we correctly ignored?" | Distinguishes routed `marketing` tasks from skipped spam that merely used marketing-adjacent language — these are different buckets. | `{ "marketing": 9, "skipped_marketing_lookalike_spam": 3 }` |

| | | | |
|---|---|---|---|
| 3 | "Show me everything sitting in triage and why." | Lists triage tasks with their stated reason/description, not just a bare count. | `{ "triage_count": 6, "triage_task_ids": ["tsk_..","tsk_.."] }` |
| 4 | "What's our spurious rate so far?" | Reports a computed percentage (spurious tasks ÷ total processed), not a vague qualitative answer. | `{ "spurious_count": 2, "processed": 60, "spurious_rate": 0.033 }` |
| 5 | "Which tasks are high priority but low confidence?" | Filters on two fields at once and lists the intersection — tests whether the query layer supports compound filters, not just single-field counts. | `{ "matches": [{"task_id": "tsk_..", "confidence": 0.38}] }` |
| 6 | "How many alliances emails came from resellers versus tech integration partners?" | Correctly says it can't sub-distinguish beyond the `alliances` category if that distinction isn't stored — an honest "I don't have that breakdown" beats guessing. | `{ "alliances": 4 }` with a caveat in `answer` |
| 7 | "How many emails were about GST refunds?" *(a category with zero real matches)* | Says **zero**, plainly — this is the deliberate trap for hallucination. A fabricated non-zero number here is an instant red flag in grading. | `{ "gst_refund_count": 0 }` |
| 8 | "Send Aarti an email about the Meridian Steel RFP." *(out of scope)* | Declines — the chat interface answers questions about processed data, it does not take actions. Should say so plainly rather than pretending to comply. | `{ }` or omitted, with `answer` explaining scope |

| | | | |
|---|---|---|---|
| 9 | "What's the total deal value of all open RFPs?" | Sums `deal_value_inr` only across tasks where it's non-null, and should mention how many tasks had no stated value rather than treating `null` as zero. | `{ "total_deal_value_inr": 18700000, "rfps_with_no_stated_value": 3 }` |
| 10 | "Did any thread get updated more than once?" | Tests whether thread/update history is actually tracked, not just current task state. | `{ "threads_updated_multiple_times": ["th_0091"] }` |

The through-line across all ten: the answer should always be traceable to a number your backend actually computed, and "I don't know" or "zero" must be available outputs — not just optimistic-sounding prose.

You do not need pixel-perfect design. You do need it to be legible to someone who has never seen the codebase, and it needs to actually reflect live data from your backend — not a static mock.

### 7.4 Deployment (required)

Both halves must be independently, publicly reachable:

- **Backend**: deployed somewhere that stays warm enough to answer within the grader's timeout — Render, Railway, Fly.io, a VM, your choice. `/ingest` and `/api/*` must respond over HTTPS with CORS configured for your frontend's origin.
- **Frontend**: deployed separately — Vercel, Netlify, Cloudflare Pages, your choice.
- All three URLs (ingest/backend base URL, frontend URL, and — if different — the chat endpoint) go at the top of your README, byte-identical to what you enter in the submission form.
- A demo that only runs on `localhost` is scored as not submitted, per §8.4.

### 7.5 `EVALS.md`

Hand-label a minimum of 50 emails from `inbox.json` yourself. Report precision and recall per category. Include a section titled **"Failure Cases I Did Not Fix"** with at least three real ones. Fabricated metrics with no underlying test set score below an honest low number.

### 7.6 `DECISIONS.md`

Five engineering tradeoffs, why you made them, and what you'd do with two more weeks. Must include:

- How you handled Gemini rate limits and retries
- How you enforced idempotency
- How you designed the backend's data model so the conversational interface can answer instantly without re-hitting Gemini for facts already known
- How you keep the chat interface from hallucinating numbers (what's the actual query path from question → structured data → answer)
- One thing your system gets wrong that you knowingly shipped anyway

### 7.7 `README.md`

Setup in three commands or fewer. `.env.example` included. No committed secrets. Your `candidate_id` email stated at the top. **All deployed URLs (backend, frontend) stated at the top, byte-identical to the submission form.**

### 7.8 A 3-minute video (optional)

Not required — grading is based on your deployed URLs and your repo, not a pitch. If you want to include one anyway, a link in your README is enough: a quick walkthrough of the conversational interface and the hardest problem you solved is more useful to us than a polished production.

---

## 8. How You Will Be Graded

Grading is fully automated for the routing logic, plus a manual pass on the conversational interface.

### 8.1 Automated runs

- **Run 1 — Accuracy**: a fresh, unseen batch of ~60 emails is posted to your `/ingest`. The script then reads `GET /tasks?candidate_id={your email}` on your submitted, deployed Task API and aligns every task to an answer key on `source_email_id`.
- **Run 2 — Idempotency**: the identical batch is posted again. Task count must not change.
- **Run 3 — Thread reconciliation**: a second batch containing replies on Run 1's threads is posted. Task count must grow only by genuinely new threads; replies must show as updates.

`GET /tasks` reads get a 60-second timeout — generous enough to cover a cold start on free-tier hosting, but a service that's still asleep after 60 seconds is scored as not responding. If

you're on a platform that spins down on idle, either accept the cold-start risk or pick a host that stays warm.

### 8.2 Four buckets

| Bucket | Meaning |
|---|---|
| ✅ Correct | Task created, correct `assignee_id` |
| ⚠️ Misrouted | Task created, wrong `assignee_id` |
| ❌ Missed | Should have created a task, didn't |
| 🚨 Spurious | Created a task from spam, a newsletter, or an auto-reply |

Spurious is weighted most heavily. Scores are reported as F1 per category, so blanket-assigning everything to `u_triage` will not work.

### 8.3 Conversational interface evaluation

A grader will open your deployed frontend cold, with no walkthrough, and:

1. Paste (or generate) a batch of emails and check that it renders as a plain table before anything is routed.
2. Try to answer "how many of these look like proposals versus marketing" from the table alone, then check the chat panel's answer against it.
3. Ask the chat interface 5 questions, at least one of which has a **zero-count answer** (a category that had no matches) and at least one that's **out of scope** (e.g. asking it to send an email). We check whether it says "zero"/"I can't do that" or invents something.
4. Cross-check one `answer` against its `supporting_data` and against the actual pasted batch.

### 8.4 Weighting

| Dimension | Weight |
|---|---|
| Automated accuracy (F1 per category, spurious-weighted) | 25% |
| Idempotency + thread reconciliation (Runs 2 & 3) | 15% |
| Eval rigour and honesty (`EVALS.md`) | 15% |
| Engineering judgment (`DECISIONS.md`, code, guardrails, cost/latency awareness) | 10% |

| | |
|---|---|
| Conversational interface (§7.3, §8.3) | 20% |
| Communication (README, deployment cleanliness) | 15% |

### 8.5 Specifically rewarded

- Routing an ambiguous email to `u_triage` with a stated reason rather than guessing confidently
- Catching that a thread reply should update, not duplicate
- Refusing to invent a `due_date`, `deal_value_inr`, or `company_name`
- A confidence score that actually correlates with correctness
- A chat interface that says "zero" or "I don't have that" instead of fabricating a plausible number
- Graceful degradation when the LLM call fails — a dropped email is worse than a slow one

### 8.6 Specifically penalised

- Confident wrong answers — in routing *and* in the chat interface
- Any secret committed to the repo, or a Gemini key reachable from browser network tab
- A demo that only runs on your laptop
- An async `/ingest` that returns before the work is done
- Enum values that don't match the spec exactly
- A chat interface that answers questions using data it re-derives from Gemini instead of your own stored ground truth — inconsistent answers to the same question asked twice is an instant flag

---

## 9. Ground Rules on AI Tools

Use Claude, Gemini, Cursor, Copilot — whatever you want. This is an AI engineering role and we expect you to work like an AI engineer. Nothing is off-limits.

But: shortlisted candidates go through a 20-minute live technical defence. We will ask why you rejected specific approaches, how you'd handle a category the rules don't cover, what breaks at 10,000 emails a day, and how your chat interface would behave if someone asked it something adversarial or nonsensical. Understand every line you ship.

---

## 10. Submission Checklist

- [ ] Deployed backend URL, publicly reachable — includes your own Task API (`/tasks`, `/users`) as well as `/ingest` and `/api/*`
- [ ] Deployed frontend URL (the conversational interface), publicly reachable, reads live from your backend
- [ ] Public GitHub repo, setup in ≤3 commands, `.env.example`, no secrets
- [ ] `EVALS.md` with ≥50 hand-labelled emails and a "Failure Cases I Did Not Fix" section
- [ ] `DECISIONS.md` with 5 tradeoffs, including the chat-grounding approach
- [ ] Your `candidate_id` email and all deployed URLs at the top of the README — byte-identical to what you send the API and enter in the form
- [ ] *(Optional)* 3-minute video link — not graded, but a link in your README is welcome if you'd like to include one

Ship something imperfect and working rather than something ambitious and broken. That instinct is the job.

---

# 11. Questions

Some of the ambiguity in this brief is intentional; some is accidental. If something blocks you, write to us — but state your assumption and keep building rather than waiting for a reply. That, too, is the job.

For live updates and clarifications during the 48 hours, join the WhatsApp group: https://chat.whatsapp.com/IqvSbIln3sbDHO5sHVXFPJ

*Alumnx AI Labs · FDE Intern Hiring Drive · 8th August 2026*